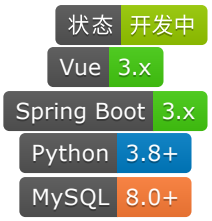
灵境行者 (Spirit Journey Rider)



现代化的智能骑行应用平台

快速开始 • 功能特性 • 技术架构 • 开发指南 • API文档

📖 项目简介

灵境行者是一个现代化的智能骑行应用平台，致力于为骑行爱好者提供全方位的骑行体验。项目采用前后端分离的微服务架构，集成了路径规划、装备推荐、社区互动等核心功能，为用户打造一站式的骑行服务生态。

✨ 项目亮点

- 🎯 **智能路径规划**: 基于高德地图API的多策略路径优化
- 🛒 **装备推荐系统**: 多平台数据爬取与智能推荐
- 🏗️ **微服务架构**: 前后端分离，服务解耦，易于扩展
- 📱 **现代化界面**: Vue3 + 响应式设计，优秀的用户体验
- 🔒 **安全可靠**: JWT认证，参数验证，全局异常处理

🏗️ 项目架构

```
灵境行者项目
├── vue-cycling-app/     # 前端Vue3应用
├── java-backend/        # Java主后端服务 (Spring Boot)
├── python-backend/      # Python爬虫服务 (Flask)
└── start-java.ps1      # Java服务启动脚本
```

架构设计

- **前端层**: Vue3 + Element Plus + 高德地图JS API
- **API网关层**: Java Spring Boot (主服务) + Python Flask (爬虫服务)
- **数据层**: MySQL 8.0 (主数据库) + 内存缓存
- **外部服务**: 高德地图API、电商平台数据源

🚀 核心功能

🗺️ 智能路径规划

- 支持驾车、步行、公交、骑行四种交通方式
- 多种路径策略：速度优先、费用优先、距离优先
- 实时路况信息和路径优化
- 地理编码和逆地理编码服务

🛒 装备推荐系统

- 多平台骑行装备数据爬取（淘宝、京东等）
- 智能搜索和价格比较
- 个性化装备推荐
- 价格趋势分析
- **数据可视化分析**:
    - 价格历史趋势图：多平台价格变化对比
    - 品牌市场份额分析：各品牌占有率统计
    - 价格分布统计：不同价格区间产品分布
    - 品牌综合评分雷达图：多维度品牌对比

👥 用户系统

- 用户注册、登录、权限管理
- 个人骑行记录管理
- 路线收藏和评分
- 用户反馈系统

📱 现代化界面

- 响应式设计，支持多设备访问
- 直观的地图交互界面
- 实时数据展示和可视化
- 优秀的用户体验设计

🛠️ 技术栈

前端技术

- **框架**: Vue 3.x + Composition API
- **UI库**: Element Plus
- **地图**: 高德地图JavaScript API
- **图表库**: Chart.js + Vue-ChartJS (数据可视化)

- **路由**: Vue Router 4.x
- **状态管理**: Pinia
- **构建工具**: Vite
- **HTTP客户端**: Axios

后端技术

- **Java服务**: Spring Boot 3.x + Spring Data JPA
- **Python服务**: Flask + SQLAlchemy
- **数据库**: MySQL 8.0
- **缓存**: 内存缓存（可扩展Redis）
- **爬虫**: Selenium + BeautifulSoup
- **API文档**: Spring Boot Actuator

开发工具

- **版本控制**: Git
- **IDE**: VS Code / IntelliJ IDEA
- **包管理**: npm (前端) + Maven (Java) + pip (Python)
- **容器化**: Docker (可选)

🚀 快速开始

📊 数据库架构

核心数据表

- **用户系统**: `users` - 用户信息、认证、权限管理
- **骑行路线**: `cycling_routes` - 路线规划、收藏、评分
- **骑行记录**: `cycling_records` - 骑行轨迹、统计数据
- **装备管理**: `equipment` - 装备信息、价格、规格
- **价格监控**: `equipment_price_history` - 价格历史、趋势分析
- **用户互动**: `route_likes`, `route_favorites`, `route_ratings`

数据库特性

- **高性能索引**: 优化的查询索引设计
- **全文搜索**: 装备和路线的全文检索
- **分区表**: 大数据量表的分区优化
- **外键约束**: 完整的数据一致性保证
- **软删除**: 数据安全的软删除机制

📋 环境要求

组件	版本要求	说明
Node.js	16.x+	前端开发环境
Java	JDK 17+	主后端服务
Python	3.8+	爬虫服务
MySQL	8.0+	主数据库
Chrome	最新版	爬虫WebDriver

⚡ 一键启动 (推荐: Docker Compose)

最简单的运行方式是使用我们提供的 Docker 一键启动脚本。只需确保您的系统上安装了 Docker Desktop：

```
# 1. 克隆项目
git clone <repository-url>
cd site

# 2. 双击运行启动脚本 (Windows)
# 找到项目根目录下的 start_docker.bat 并双击运行。
# 脚本会自动拉取所需的镜像、初始化数据库，并分别在 80、5000 和 8080 端口启动各个服务。

# 或者使用命令行手动运行 (Linux/Mac)
docker-compose up -d --build
```

初始系统管理员账号：

- 用户名：`root`
- 密码：`123456`

💻 手动分步启动 (或使用本地一键脚本)

如果您不使用 Docker，可以使用以下方式本地启动：

推荐：使用本地一键启动脚本
双击项目根目录下的 `start_local.bat`，脚本将自动为您开启 3 个命令行窗口，分别运行 Java 后端、Python 后端和 Vue 前端（请确保您已经启动了本地的 MySQL，并且配置好了所需的开发环境：Java、Python、Node）。

或者完全手动分步执行：

```
# 1. 数据库初始化
mysql -u root -p -e "CREATE DATABASE ljxz CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -u root -p ljxz < database/complete_init.sql

# 2. 启动所有服务（推荐使用多个终端窗口）
# 终端1: Java后端
cd java-backend && mvn spring-boot:run

# 终端2: Python后端
cd python-backend && pip install -r requirements.txt && python app.py

# 终端3: 前端应用
cd vue-cycling-app && npm install && npm run dev
```

🔧 详细安装步骤

▶ 点击展开详细安装指南

配置说明

高德地图API配置

- 在 `java-backend/src/main/resources/application.yml` 中配置高德地图API密钥
- 在 `vue-cycling-app/src/config/map.js` 中配置前端地图API密钥

数据库配置

- Java后端: `application.yml` 中的数据库连接配置
- Python后端: `config.py` 中的数据库连接配置

📚 文档导航

📖 核心文档

文档	描述	链接
🎨 前端开发指南	Vue3应用开发文档	README
☕ Java后端文档	Spring Boot服务文档	README
🐍 Python爬虫文档	Flask爬虫服务文档	README

📋 配置文档

文档	描述	链接
🔌 API接口文档	完整的API端点说明	API_ENDPOINTS
🗄️ 数据库配置	数据库安装配置指南	DATABASE_SETUP

文档	描述	链接
🗺️ 地图API配置	高德地图API配置说明	MAP_CONFIG

🛠️ 开发文档

文档	描述	链接
🏗️ 架构设计	系统架构和设计模式	ARCHITECTURE
🔄 部署指南	生产环境部署文档	DEPLOYMENT
🧪 测试指南	测试策略和用例	TESTING
🤝 贡献指南	代码贡献规范	CONTRIBUTING

🌐 服务端口

服务	端口	访问地址	状态检查
🎨 **前端应用**	5173	http://localhost:5173	直接访问
☕ **Java后端**	8080	http://localhost:8080	健康检查
🐍 **Python后端**	5000	http://localhost:5000	健康检查
🗄️ **MySQL数据库**	3306	localhost:3306	`mysql -u root -p`

🔗 API文档

- **Java API文档**: http://localhost:8080/swagger-ui.html (开发中)
- **Python API文档**: http://localhost:5000/docs (开发中)
- **接口测试工具**: Postman Collection

🔧 开发指南

📝 代码规范

技术栈	规范标准	工具
Vue3前端	Vue3官方风格指南	ESLint + Prettier
Java后端	Spring Boot最佳实践	Checkstyle + SpotBugs
Python后端	PEP 8	Black + Flake8

🌿 分支管理

```
main                    # 主分支，生产环境代码
├── develop            # 开发分支，集成测试
├── feature/xxx        # 功能分支
├── hotfix/xxx         # 热修复分支
└── release/xxx        # 发布分支
```

📋 提交规范

```
# 提交格式
<type>(<scope>): <subject>

# 示例
feat(route): 添加骑行路径规划功能
fix(auth): 修复用户登录验证问题
docs(readme): 更新项目文档
style(ui): 优化地图界面样式
refactor(api): 重构路径规划API
test(unit): 添加用户服务单元测试
```

🧪 测试策略

层级	前端	Java后端	Python后端
单元测试	Vitest	JUnit 5	pytest
集成测试	Cypress	Spring Boot Test	pytest + requests
E2E测试	Playwright	TestContainers	Selenium

🚀 开发流程

1. **需求分析** → 2. **技术设计** → 3. **编码实现** → 4. **单元测试** → 5. **集成测试** → 6. **代码审查** → 7. **部署发布**

🤝 贡献指南

1. Fork 项目
2. 创建功能分支 (`git checkout -b feature/AmazingFeature`)
3. 提交更改 (`git commit -m 'Add some AmazingFeature'`)
4. 推送到分支 (`git push origin feature/AmazingFeature`)
5. 开启 Pull Request

📚 文档导航

核心文档

- 📖 项目总览 - 项目介绍和快速开始
- 📐 架构设计 - 系统架构和技术选型
- 🛠️ 开发指南 - 开发环境和编码规范
- 🚀 部署文档 - 生产环境部署指南

前端文档

- 🎨 Vue前端应用 - 前端应用详细文档
- 📱 API接口文档 - 前端API调用指南

后端文档

- ☕ Java后端服务 - Spring Boot主服务文档
- 🐍 Python爬虫服务 - Flask爬虫服务文档

数据库文档

- 🗄️ 数据库设计 - 完整的数据库设计文档
- ⚙️ 数据库设置 - 数据库安装配置指南
- 📊 初始化脚本 - 数据库建表脚本

技术文档

- 🔒 安全策略 - 安全设计和最佳实践
- 📋 API规范 - RESTful API设计规范

📄 许可证

本项目采用 MIT 许可证 - 查看 LICENSE 文件了解详情。

📞 联系我们

角色	姓名	联系方式
🎓 **项目负责人**	陈冠衡	925342921@qq.com
🏫 **所属院校**	河北经贸大学	-
📍 **项目地址**	GitHub仓库	-

🤝 参与贡献

欢迎提交 Issue 和 Pull Request！

报告Bug • 功能建议 • 贡献代码

🚴 灵境行者 - 让每一次骑行都成为一场精彩的旅程！ ✨

Built with ❤️ by 河北经贸大学团队